

Python Programlama

Ders 10

Ali Mertcan KOSE Msc.

`amertcankose@ticaret.edu.tr`

İstanbul Ticaret Üniversitesi



İSTANBUL TİCARET
ÜNİVERSİTESİ

NumPy, bilimsel hesaplama ve veri analizi için kullanılan, “Numerical Python” (Sayısal Python) ifadesinin kısaltması olan bir Python modülüdür.

NumPy şunları sağlar:

- Hızlı ve verimli veri depolama ve işleme için çok boyutlu diziler (veri yapıları).
- Verileri manipüle etmek için kullanışlı fonksiyonlar.
- Programlara veri aktarma ve programlardan veri alma araçları.

NumPy'nin veri analizi için sunduğu işlevler şunlardır:

- Veri manipülasyonu için dizi (array) işlemleri.
- Sıralama, benzersiz (unique) değerler ve küme işlemleri gibi yaygın algoritmalar.
- Betimleyici istatistikler, veri toplama ve özetleme işlemleri.
- Veri setlerinin birleştirilmesi ve eklenmesi.
- Döngüler yerine koşullu mantığın dizi ifadeleriyle ifade edilmesi.

NumPy İşlevselliğini Kullanma

NumPy paketini kullanmak için, öncelikle daha önce olduğu gibi içe aktarılması (import edilmesi) gerekir.

`import numpy as np` -> Paket içindeki işlevselliği kullanmak istediğimizde, şu sözdizimini kullanacağız: `np.functionName(...)`

NumPy Veri Yapısı: ndarray

Bir veri yapısında birden fazla değeri bir arada depolar.

Bellekte sabit boyutlu bir dizi olarak bulunur ve aynı türden verileri içerir. Listelerden farklı olarak, tüm öğeler aynı türde olmalıdır, bu nedenle bir dizi farklı türleri içeremez.

Bir NumPy dizisinin boyutu oluşturulduğunda sabittir; bir diziyi büyütme veya küçültme mümkün değildir.

Eleman ekleme, ekleme veya silme işlemleri yapılamaz.

NumPy dizileri değiştirilebilir (mutable) olduğundan, dizide depolanan değerler değiştirilebilir.

Döngü kullanmadan, eleman bazında işlemlere olanak tanıyan yerleşik işlevselliklere sahiptir.

NumPy Dizi (Array) Özellikleri

Bir NumPy dizisi, aynı türdeki değerlerin bir satırı veya ızgarasıdır ve sıfırdan küçük olmayan tam sayılardan oluşan bir “tuple” ile indekslenir. Her dizinin aşağıdaki özellikleri vardır:

- `dtype` → Dizideki veri türü
- `ndim` → Boyut sayısı
- `shape` → Her bir boyutun boyutu
- `size` → Dizideki toplam eleman sayısı

ndarray oluşturmak için fonksiyonlar

- `array()`: Verilen değerler dizisiyle bir dizi (array) oluşturur.
- `arange()`: Belirtilen aralıktaki değerlerle bir dizi oluşturur.
- `linspace()`: Belirtilen aralıkta, n adet eşit aralıklı değerle bir dizi oluşturur.
- `empty()`: Belirtilen şekle sahip, rastgele (boş) öğelerle dolu bir dizi oluşturur.

ndarray oluşturmak için fonksiyonlar

- `zeros()`: Belirtilen şekle sahip, sıfırlarla dolu bir dizi oluşturur.
- `ones()`: Belirtilen şekle sahip, birlerle dolu bir dizi oluşturur.
- `rand()` `randint()`: Belirtilen şekle sahip, rastgele üretilmiş değerlerle bir dizi oluşturur.

NumPy Dizileri (Arrays) Oluşturma

NumPy dizilerini Python listelerinden başlatabilir ve elemanlara köşeli parantezler kullanarak erişebiliriz. Bir listeden dizi oluşturmak için `array()` komutunu kullanabilir ve listeyi bir parametre olarak geçirebiliriz.

Örnek Bir boyutlu bir dizi oluşturma:

```
import numpy as np  
  
z = np.array([4,7,6,9,2])  
z Out[13]: array([4, 7, 6, 9, 2])  
print(z)
```

Özniteliklerle İlgili Örnekler

```
r = np.array([87,33,5,16,9,17])
```

```
r
```

```
Out[29]: array([87, 33, 5, 16, 9, 17])
```

```
r.dtype
```

```
Out[30]: dtype('int32')
```

Özniteliklerle İlgili Örnekler

```
r.ndim
```

```
Out[31]: 1
```

```
r.shape
```

```
Out[32]: (6,)
```

```
r.size
```

```
Out[33]: 6
```

NumPy Dizileri Oluşturma

NumPy dizileri, `arange` fonksiyonu kullanılarak da oluşturulabilir. **`arange`** fonksiyonu, başlangıç, bitiş ve adım değerlerini alır ve bu değerleri içeren bir dizi oluşturur.

Örnek:

```
b = np.arange(1,10,2)
```

```
b
```

```
Out[67]: array([1, 3, 5, 7, 9])
```

NumPy Dizileri Oluşturma

```
t = np.arange(1,10)
```

```
t
```

```
Out[65]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

linspace()

Fonksiyon, eşit aralıklı ondalıklı (float) değerler oluşturur. Başlangıç, bitiş ve değer sayısını parametre olarak alır. Belirtilen aralıkta verilen sayıda değer üretir.

linspace()

```
In [31]: np.linspace?
         np.linspace(-50, 50, 11)

Out[31]: array([-50., -40., -30., -20., -10.,   0.,  10.,  20.,  30.,  40.,  50.])

In [33]: np.linspace(-10, 10)

Out[33]: array([-10.    , -9.5918, -9.1837, -8.7755, -8.3673, -7.9592,
        -7.551  , -7.1429, -6.7347, -6.3265, -5.9184, -5.5102,
        -5.102  , -4.6939, -4.2857, -3.8776, -3.4694, -3.0612,
        -2.6531, -2.2449, -1.8367, -1.4286, -1.0204, -0.6122,
        -0.2041,  0.2041,  0.6122,  1.0204,  1.4286,  1.8367,
         2.2449,  2.6531,  3.0612,  3.4694,  3.8776,  4.2857,
         4.6939,  5.102  ,  5.5102,  5.9184,  6.3265,  6.7347,
         7.1429,  7.551  ,  7.9592,  8.3673,  8.7755,  9.1837,
         9.5918, 10.    ])
```

Figure 1: linspace()

Özel Diziler Oluşturma

Başlangıç değerlerine sahip diziler oluşturmak için daha önce array ve arange fonksiyonlarını kullandık. Ayrıca, diğer varsayılan değerlerle (örneğin, ones, zeros) diziler oluşturmak için kullanabileceğimiz özel fonksiyonlar da mevcuttur.

Özel Diziler Oluşturma

Function	Output
array	Bir NumPy dizi (array) nesnesi oluşturur.
arange	Belirtilen aralıkta (başlangıç, bitiş, adım) değerlerle bir NumPy dizisi oluşturur.
ones	Belirtilen şekle sahip, tüm elemanları bir olan bir NumPy dizisi oluşturur.
zeros	Belirtilen şekle sahip, tüm elemanları sıfır olan bir NumPy dizisi oluşturur.

1D-Boyutlu Diziler Oluşturma: zeros() ve ones()

```
In [1]: import numpy as np

In [2]: np.zeros(10)
Out[2]: array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])

In [4]: np.ones(10)
Out[4]: array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

Figure 2: zeros() ve ones()

Rastgele Diziler Oluşturma

NumPy'de, rastgele diziler oluşturmak için kullanılan `random` adlı bir alt modül bulunmaktadır.

Örnek:

`rand(n)`: 0 ile 1 arasında n adet rastgele değer içeren bir dizi oluşturur.

`randint()`: `low` (dahil) ile `high` (hariç) arasında n adet rastgele tamsayı değeri içeren bir dizi oluşturur.

Rastgele Diziler Oluşturma

```
import numpy as np  
r_vals = np.random.rand(10)  
print(r_vals)  
r_vals = np.random.randint(1,26,12)  
print(r_vals)
```

NumPy Dizilerine Erişim ve Güncelleme

Erişim elements:

`z[0] -> 4`

`z[3] -> 9`

Güncelleme elements:

`z[2] = 5`

`z`

Out[19]: `array([4, 7, 5, 9, 2])`

NumPy Dizilerini Dilimleme (Slicing)

Listelerde olduğu gibi, NumPy dizilerinin belirli bölümlerine başlangıç:bitis:adım sözdizimini kullanarak erişebiliriz.

Hatırlatma:

- bitis değeri belirtilen eleman dahil edilmez (hariç tutulur).
- Herhangi bir değer atlanırsa, varsayılan değerler şunlardır:
- başlangıç: 0
- bitis: dizinin sonu
 - adım: 1

NumPy Dizilerini Dilimleme (Slicing)

Örnekler:

z

Out[22]: array([4, 7, 5, 9, 2])

z[1:4:2]

Out[21]: array([7, 9])

z[1:4]

Out[23]: array([7, 5, 9])

z[1::2]

Out[24]: array([7, 9])

z[:4]

Out[25]: array([4, 7, 5, 9])